

"""

=====

=====

ANALISADOR DE PROTOCOLO DE COERÊNCIA (PIC/SFT)

Projeto 2 — Fase 1: Dados Sintéticos | Fase 2: Dados Hiper-Realistas

Autor: Kimi (Moonshot AI) | Data: 2026-07-28

Framework: Princípio da Informação Consciente (PIC) + Teoria do Campo Sinérgico (SFT)

=====

=====

MÉTRICAS PIC:

Φ (Informação Integrada) = $\text{Coerência} / (1 + 0.1 * \text{Entropia})$

S (Campo Sinérgico) = $\lambda * |d\Phi/dt|$

Entropia = Shannon espectral média

Complexidade = Desvio padrão das entropias entre canais

Coerência = Coerência média inter-canal na banda alfa (8-13 Hz)

PARÂMETROS:

$\lambda = 0.8$ (constante de acoplamento sinérgico)

Janela = 2s, Passo = 1s

USO:

python analisador_pic_coerencia.py

Para usar com dados reais do PhysioNet EEGMMIDB:

1. Baixe os arquivos .edf de <https://physionet.org/content/eegmmidb/1.0.0/>
2. Substitua a seção de geração de dados pela leitura dos arquivos EDF
3. Ajuste fs_real para 160 Hz e N_ch_real para 64

=====

=====

"""

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from scipy import signal
```

```
from scipy.stats import entropy
```

```
from scipy.spatial.distance import pdist, squareform
```

```
import warnings
```

```
warnings.filterwarnings('ignore')
```

```
# =====
```

```
# PARÂMETROS GLOBAIS PIC/SFT
```

```
# =====
```

```
lambda_sft = 0.8    # Constante de acoplamento sinérgico
threshold_phi = 0.6  # Limiar de ignição consciente
```

```
# =====
```

```
# FUNÇÕES UTILITÁRIAS
```

```
# =====
```

```
def pink_noise(n_samples, n_channels, fs):
    """Gera ruído 1/f (pink noise) realista para EEG."""
    noise = np.zeros((n_channels, n_samples))
    for ch in range(n_channels):
        white = np.random.randn(n_samples)
        fft_w = np.fft.rfft(white)
        freqs = np.fft.rfftfreq(n_samples, 1/fs)
        freqs[0] = 1e-10
        pink_f = 1 / np.sqrt(freqs)
        pink_f[0] = 0
        noise[ch] = np.fft.irfft(fft_w * pink_f, n=n_samples)
    return noise
```

```
def calculate_pic_metrics(eeg_data, fs, window_sec=2.0, step_sec=1.0,
                          lambda_sft=0.8, sm_channels=None):
```

```
    """
```

```
    Calcula métricas PIC/SFT em janelas deslizantes.
```

```
    Parâmetros:
```

```
    eeg_data: ndarray (n_channels, n_samples)
    fs: frequência de amostragem (Hz)
    window_sec: duração da janela em segundos
    step_sec: passo entre janelas em segundos
    lambda_sft: constante de acoplamento sinérgico
    sm_channels: lista de índices de canais sensorimotores (opcional)
```

```
    Retorna:
```

```
    dict com: phi, entropy, coherence, complexity, S_field, t_windows
```

```
    """
```

```
n_channels, n_samples = eeg_data.shape
window_samples = int(window_sec * fs)
step_samples = int(step_sec * fs)
n_windows = (n_samples - window_samples) // step_samples + 1

phi = np.zeros(n_windows)
entropy_vals = np.zeros(n_windows)
coherence = np.zeros(n_windows)
```

```

complexity = np.zeros(n_windows)
S_field = np.zeros(n_windows)

# Se não especificado, usar todos os canais para coerência
if sm_channels is None:
    sm_channels = list(range(n_channels))

for w in range(n_windows):
    start = w * step_samples
    end = start + window_samples
    win_data = eeg_data[:, start:end]

    # 1. Entropia espectral média
    entropies = []
    for ch in range(n_channels):
        f, psd = signal.welch(win_data[ch], fs, nperseg=min(256, window_samples//2),
                              noverlap=min(192, window_samples//4))
        psd_norm = psd / np.sum(psd)
        psd_norm = psd_norm[psd_norm > 0]
        entropies.append(entropy(psd_norm, base=2))
    entropy_vals[w] = np.mean(entropies)

    # 2. Coerência média (banda alfa: 8-13 Hz)
    coh_vals = []
    for i in range(len(sm_channels)):
        for j in range(i+1, len(sm_channels)):
            f, Cxy = signal.coherence(win_data[sm_channels[i]], win_data[sm_channels[j]],
                                      fs, nperseg=min(256, window_samples//2))
            alfa_mask = (f >= 8) & (f <= 13)
            if np.any(alfa_mask):
                coh_vals.append(np.mean(Cxy[alfa_mask]))
    coherence[w] = np.mean(coh_vals) if coh_vals else 0

    # 3.  $\Phi$  (Informação Integrada)
    phi[w] = coherence[w] / (1 + 0.1 * entropy_vals[w])

    # 4. Complexidade
    complexity[w] = np.std(entropies)

# 5. Campo Sinérgico  $S = \lambda |d\Phi/dt|$ 
S_field[0] = 0
for w in range(1, n_windows):
    S_field[w] = lambda_sft * abs(phi[w] - phi[w-1]) / step_sec

```

```

t_windows = np.arange(n_windows) * step_sec + window_sec/2

return {
    'phi': phi,
    'entropy': entropy_vals,
    'coherence': coherence,
    'complexity': complexity,
    'S_field': S_field,
    't_windows': t_windows,
    'n_windows': n_windows
}

def detect_phi_ignition(phi, threshold=0.6):
    """Detecta o momento de ignição ( $\Phi$  cruza o limiar pela primeira vez)."""
    above = np.where(phi > threshold)[0]
    return above[0] if len(above) > 0 else -1

def classify_state(phi, entropy, coherence, complexity):
    """
    Classificador simples de estado de consciência baseado em thresholds.
    Retorna: 0=Descoerente, 1=Coerente, 2=Supercoerência
    """
    states = np.zeros(len(phi), dtype=int)
    for i in range(len(phi)):
        if phi[i] < 0.4 and entropy[i] > 3.5:
            states[i] = 0 # Descoerente
        elif phi[i] > 0.6 and coherence[i] > 0.7:
            states[i] = 2 # Supercoerência
        else:
            states[i] = 1 # Coerente
    return states

# =====
# EXEMPLO DE USO — DADOS SINTÉTICOS
# =====
if __name__ == "__main__":
    print("="*70)
    print(" ANALISADOR DE PROTOCOLO DE COERÊNCIA (PIC/SFT)")
    print("="*70)

    # Parâmetros
    fs = 256

```

```

T = 60
n_ch = 8
n_samples = fs * T

# Gerar dados sintéticos (exemplo simplificado)
eeg = pink_noise(n_samples, n_ch, fs) * 0.5

# Calcular métricas
metrics = calculate_pic_metrics(eeg, fs, lambda_sft=lambda_sft)

print(f"\n  $\Phi$  médio: {np.mean(metrics['phi']):.4f}")
print(f" Coerência média: {np.mean(metrics['coherence']):.4f}")
print(f" Entropia média: {np.mean(metrics['entropy']):.4f} bits")
print(f" |S| máximo: {np.max(metrics['S_field']):.4f}")

ignition = detect_phi_ignition(metrics['phi'], threshold_phi)
if ignition >= 0:
    print(f" Ignição  $\Phi$  detectada em t={metrics['t_windows'][ignition]:.1f}s")
else:
    print(f" Ignição  $\Phi$  não atingiu limiar {threshold_phi}")

```